`scripts/check-commit-docs-exists.sh` — User Guide

Last verified: 2026-05-18 (1.2.30-1050 tooling cycle)
Inheritance: HelixConstitution §11.4.x (commit-references-resolve mandate) + Lava §6.AD-debt closure (CM-COMMIT-DOCS-EXISTS)

Overview

Per-commit reference resolver. Scans commit message bodies for file path citations and verifies each one resolves to a real file at HEAD. A commit message that cites "evidence at `.lava-ci-evidence/X.json`" when X.json was never created (or was deleted later) is a §6.J spirit violation — the claim is unfalsifiable because the referenced evidence does not exist.

Usage

```
# Default — scan HEAD only
bash scripts/check-commit-docs-exists.sh

# Scan a range of commits
bash scripts/check-commit-docs-exists.sh HEAD~5..HEAD

# Scan all unpushed commits (recommended pre-push)
bash scripts/check-commit-docs-exists.sh '@{u}..HEAD'

# Env-var alternative (used by hermetic tests + the sweep wrapper)
LAVA_COMMIT_RANGE='HEAD~3..HEAD' bash scripts/check-commit-docs-
        exists.sh
LAVA_REPO_ROOT=/path/to/synthetic bash scripts/check-commit-docs-
        exists.sh
```

Exit codes

Code	Meaning
0	Every referenced path resolves (exact or fuzzy-basename)

Code	Meaning
1	At least one orphan reference; commit SHA + path printed to stderr

What counts as a path reference

A token in a commit message body is considered a path reference when ALL of these hold:

1. It contains at least one `/` (directory separator).
2. It ends with one of the recognized extensions: `md json sh kt go kts gradle xml yaml yml toml conf txt tsv`.
3. It starts with one of the canonical project-root prefixes: `docs/`, `scripts/`, `tests/`, `.lava-ci-evidence/`, `submodules/`, `constitution/`, `app/`, `core/`, `feature/`, `proxy/`, `lava-api-go/`, `tools/`, `buildSrc/`, `config/`, `keystores/`, `releases/`, `Upstreams/`, `.githooks/`.

These three conditions together filter out version IDs (`1.2.30-1050`), package names that happen to contain dots, and URLs.

Whitelist conditions (intentional skips)

- **Backtick spans** (`` `like-this` ``) are stripped before regex match. Prose markup is not a citation.
- **`~~strikethrough~~` spans** are stripped. The closure-mark convention intentionally references obsolete paths.
- **Blockquote lines** (lines starting with `> `) are dropped. §6.L cycle bodies often quote operator messages verbatim where the original might reference now-renamed paths.
- **Glob/wildcard paths** containing `*` are skipped. They are intentional non-existent path patterns.

Fuzzy basename fallback

A prose reference like `feature/search_result/SearchPageState.kt` resolves the real file at `feature/search_result/src/main/kotlin/lava/search/result/SearchPageState.kt` via this fallback:

1. Exact path check first.
2. If miss: extract the leading directory (e.g. `feature`) + the basename (`SearchPageState.kt`).
3. `find <prefix-dir> -name <basename>` excluding `build/`, `.git/`, `.claude/`.
4. If 1+ matches found, treat as resolved.

The fallback prevents pedantic false-positives on human-written short-form paths while still catching genuinely-missing files.

§6.J anti-bluff falsifiability rehearsal

1. Create an empty git commit referencing a phantom file:

   ```
   git commit --allow-empty -m "claim: see .lava-ci-evidence/
           sixth-law-incidents/2026-99-99-phantom.json"
   ```

2. Run `bash scripts/check-commit-docs-exists.sh` → expect exit 1 with the phantom path listed.
3. Either `git commit --amend` removing the reference, OR create the file, OR drop the commit.

The deliberate-phantom rehearsal proves the gate fires on real evidence gaps.

Integration

- Wired into `scripts/verify-all-constitution-rules.sh` as a standard gate (default scope: `HEAD~5..HEAD` to balance signal vs noise).
- Pre-push hooks transitively run via `scripts/check-constitution.sh`.
- Recommended pre-push scope: `@{u}..HEAD` to verify ALL unpushed commits before propagation.

Hermetic test

`tests/check-constitution/test_commit_docs_exists.sh` — 7 fixtures:

1. Commit with no path refs → pass
2. All-existing paths → pass
3. Orphan reference → reject (exit 1) + path listed
4. Stale ref inside `~~strikethrough~~` → pass (skipped)
5. Stale ref inside backticks → pass (skipped)
6. Fuzzy basename short-form → pass (resolved via find)
7. Real-repo HEAD sanity check → pass

Run:

```
bash tests/check-constitution/test_commit_docs_exists.sh
```

`Classification:` project-specific (the convention is universal per HelixConstitution; the path-prefix whitelist is Lava-specific).